

Digital Image Processing (DIP)

Project Report

ARTISTIC IMAGE STUDIO

*A MATLAB App Designer Application for Non-Photorealistic
Image Rendering & Artistic Style Transfer*

Submitted To:

Mam Amina Amir

Submitted By: Group #9

Alisha Fatima (2023-ag-10040)

Aliza Tariq (2023-ag-10041)

Laika Butt (2023-ag-10075)

Kinza Fatima (2023-ag-10072)

Maryam Zafar (2023-ag-10081)

MATLAB App Designer Project

Submitted in partial fulfillment of DIP course requirements

Spring 2026

Contents

1	Project Overview	2
2	Objectives	2
3	Tools & Technologies	3
4	Application Architecture	3
4.1	UI Components (The Screen)	3
4.2	How the Code Flows	3
5	Our Image Processing Filters	4
6	Algorithm Deep-Dive	4
6.1	Pencil Sketch — Dodge Blend Method	4
6.2	Ink Outline — Canny Edge Detection	5
6.3	Oil Painting — Bilateral + Quantization	5
6.4	Charcoal Sketch — Texture + Shadow Blend	5
6.5	Pop Art — Quantization + Outline	5
7	Before & After Results	5
7.1	Pencil Sketch Filter	5
7.2	Ink Outline Filter	6
7.3	Oil Painting Filter	6
7.4	Charcoal Sketch Filter	6
7.5	Pop Art Filter	7
7.6	Testing on Different Subjects	7
8	Code Structure & Good Practices	8
9	Conclusion	8
9.1	What We Can Add in the Future	9

1 Project Overview

Artistic Image Studio is an interactive image processing application developed using MATLAB App Designer as a Digital Image Processing (DIP) course project. The application enables users to upload portrait or general-purpose images and apply a variety of artistic rendering effects inspired by traditional art forms such as pencil sketching, charcoal drawing, ink outlining, oil painting, and pop art.

The app leverages core DIP techniques — including spatial filtering, edge detection, color quantization, and intensity transformations — to transform ordinary photographs into stylized artistic outputs, all within a clean and interactive graphical user interface (GUI). We worked hard to bring all these technical concepts into an easy-to-use interface so anyone can apply these effects without needing to know the complex mathematics behind them.

Table 1: Project Details Overview

Project Title	Artistic Image Studio
Platform	MATLAB App Designer (R2021a or later)
Domain	Digital Image Processing (DIP)
Input	JPG/JPEG/PNG image files
Output	Artistic rendered image displayed directly in the app
Interface	Simple dual-panel screen (Upload on left, Result on right)

2 Objectives

The primary objectives of this project are:

- To implement non-photorealistic rendering (NPR) techniques using MATLAB image processing toolbox functions.
- To design a user-friendly GUI that allows real-time artistic filter application.
- To apply core DIP concepts: spatial filtering, edge detection, intensity mapping, and color quantization.
- To demonstrate the transformation of input photographs into multiple distinct artistic styles.
- To provide a side-by-side preview of original and processed images for comparison.
- To explore the mathematical foundations of image transformations such as dodge blending and bilateral filtering.

3 Tools & Technologies

Our application is completely built inside MATLAB and does not require any external libraries or toolboxes beyond the standard Image Processing Toolbox.

Table 2: Tools and Technologies Used

Tool	Purpose in Our Project	Key Functions
MATLAB	The main programming and computational environment	R2021a+
App Designer	GUI development framework built into MATLAB	uifigure, uiaxes
Image Proc. Toolbox	Provides all image processing functions used	imgaussfilt, imadjust
im2double	Precision conversion between image data types	im2double
imshow	Renders images into UIAxes display panels	imshow(img, Parent)

4 Application Architecture

We organized our app's code using MATLAB's built-in App Designer structure (DIP < matlab.apps.AppBase), following MATLAB OOP conventions. It has three main layers:

4.1 UI Components (The Screen)

- **Main Window:** A 640×480 screen with a clean dark theme (background color [0.149, 0.180, 0.173]).
- **Preview Panel (Left):** Displays the original uploaded image on the left side.
- **Result Panel (Right):** Displays the artistically processed output on the right side.
- **Upload Button:** Opens a native file picker dialog for image selection (JPG/PNG).
- **Filter Dropdown:** Lists all 5 available artistic styles; triggers processing on value change.

4.2 How the Code Flows

When a user clicks "Upload", the app reads the file and saves it in `UIFigure.UserData`. This was an important step because it means the app does not have to load the image from disk again every time the user changes a filter. Image data flows: File System $\rightarrow$ `uigetfile` $\rightarrow$ `imread` $\rightarrow$ `UserData` $\rightarrow$ `im2double` $\rightarrow$ Filter Algorithm $\rightarrow$ `imshow(UIAxes2)`. When a filter is selected from the dropdown, the app takes that saved image, runs our code on it, and displays the result on the right side.

5 Our Image Processing Filters

The application implements five distinct artistic style filters. Each filter is selected via the dropdown menu and processed in real-time. Here is a summary of the techniques used behind the scenes.

Table 3: Summary of How Our Filters Work

Filter Name	Steps We Followed in the Code
Pencil Sketch	Convert to grayscale → Invert colors → Gaussian Blur → Dodge Blend → Normalization → Contrast Stretch
Ink Outline	Grayscale → Canny Edge Detection → Morphological Dilation → White Canvas Overlay
Oil Painting	Bilateral Filtering → Color Quantization (8 bins) → Median Filter → Edge Sharpening
Charcoal Sketch	Grayscale → Texture via stdfilt → Canny Edges → Shadow Blend → Contrast Adjust
Pop Art	Color Quantization (5 levels) → Canny Edge Detection → Black Outline Overlay → Contrast Boost

6 Algorithm Deep-Dive

6.1 Pencil Sketch — Dodge Blend Method

This technique replicates the look of a hand-drawn pencil sketch. The core formula is the "Color Dodge" blend mode used in digital art applications:

$output = gray ./ (1 - blurredInverted + \epsilon)$ where $\epsilon = 0.001$

- Step 1: Convert to grayscale using `rgb2gray`.
- Step 2: Invert pixel intensities (`invImg = 1 - gray`).
- Step 3: Apply strong Gaussian blur ($\sigma = 8$) to simulate pencil shading softness.
- Step 4: Apply dodge blend formula to reveal sketch-like edges.
- Step 5: Normalize with `mat2gray`, stretch contrast with `imadjust` for clean white paper effect.

6.2 Ink Outline — Canny Edge Detection

Produces a black-ink line drawing on a pure white background using the Canny edge detector with a dual-threshold approach:

- Canny thresholds `[0.08, 0.20]` detect both weak and strong edges for clean line art.
- Morphological dilation with `strel('disk', 1)` thickens lines to resemble pen strokes.
- Edges are projected onto all three RGB channels of a white (255,255,255) canvas.

6.3 Oil Painting — Bilateral + Quantization

Mimics the appearance of oil paint by smoothing while preserving edges, then reducing the color palette:

- `imbilatfilt` (bilateral filter) smooths textures while preserving hard edges ($\sigma = 0.15$, `radius = 5`).
- Color quantization to 8 discrete bins: `quantized = round(img × 7) / 7`.
- Per-channel median filter (5×5) forms cohesive "paint blobs" typical of oil paintings.
- `imsharpen` (`Radius=1.5`, `Amount=1.2`) adds crisp brush stroke boundaries.

6.4 Charcoal Sketch — Texture + Shadow Blend

Creates the rich dark texture of charcoal on paper by combining local texture information with edge strokes:

- `stdfilt` with a disk-shaped neighborhood extracts local standard deviation as a texture map.
- Shadow formula: `charcoal = (1-gray)×1.5 + texture×0.5` creates deep tonal variation.
- Edge pixels darkened further (`×0.3`) to simulate heavy charcoal strokes on detail areas.
- `imadjust` stretches output to `[0.1, 0.8]` for authentic paper-and-charcoal contrast.

6.5 Pop Art — Quantization + Outline

Inspired by Andy Warhol's pop art style, this filter creates a bold poster-like look:

- 5-level color quantization: `floor(img×5)/5` creates flat color regions.
- Canny edges (`threshold=0.1`) with dilation produce thick black contour outlines.
- `imadjust` with `stretchlim` maximizes contrast for vivid, high-saturation output.

7 Before & After Results

We tested our application on different types of pictures to make sure the filters work well on everyone. Below are the results of our testing.

7.1 Pencil Sketch Filter

7.1 Pencil Sketch Filter

PORTRAIT SAMPLE — PENCIL SKETCH EFFECT

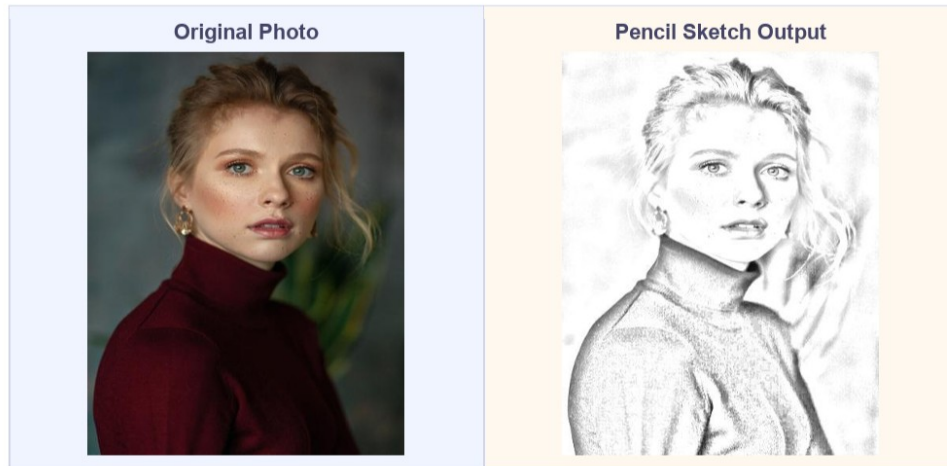


Figure 1: Testing the Pencil Sketch Effect

As seen here, our code successfully picks up small details like the hair strands and turns them into light pencil strokes. The dodge blend amplifies light-dark contrast, mimicking real pencil shading technique. The background turns clean white, just like a sketchbook.

7.2 Ink Outline Filter

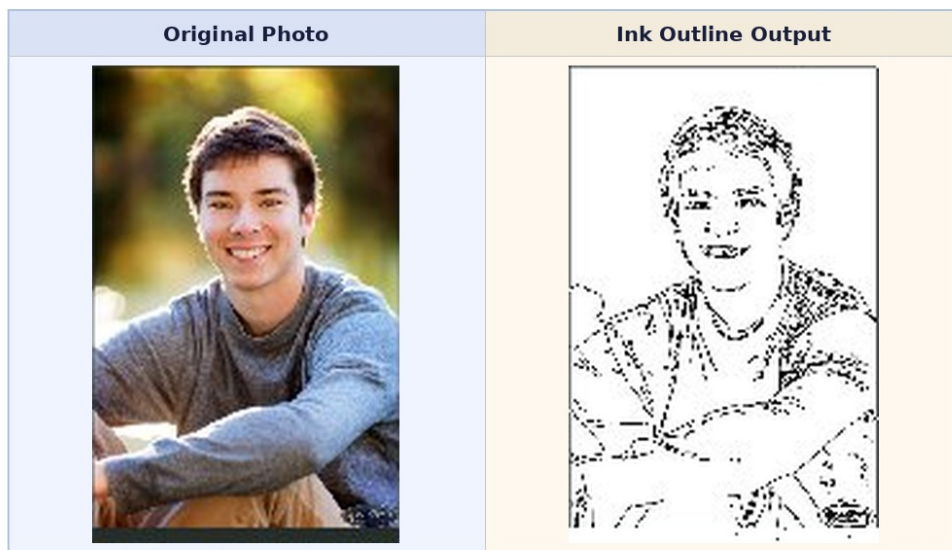


Figure 2: Testing the Ink Outline Effect

The edge detection worked perfectly here. The Canny detector with dual thresholds captures both prominent contours and subtle details. The dilation step gives ink-like weight to the strokes while the white background creates a clean line art result, making it look like a pure black-ink drawing.

7.3 Oil Painting Filter

7.3 Oil Painting Filter

PORTRAIT SAMPLE — OIL PAINTING EFFECT

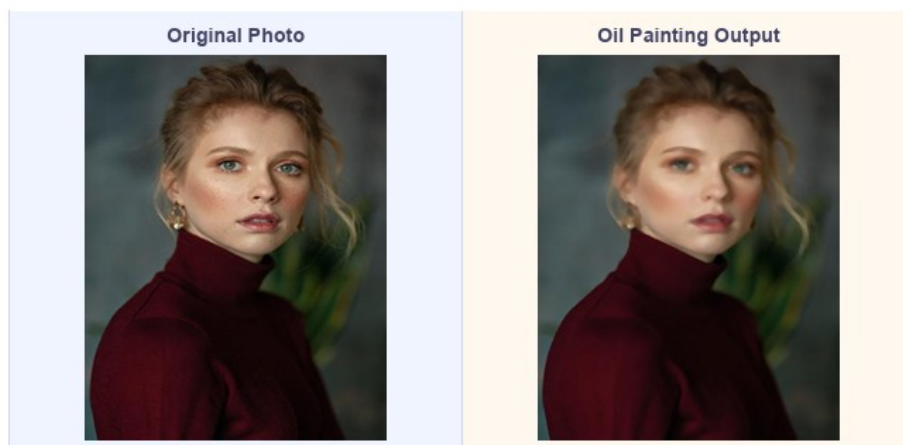


Figure 3: Testing the Oil Painting Effect

The bilateral filter effectively smooths skin texture while preserving sharp jawline and eye edges. Color quantization to 8 bins groups similar hues into distinct paint regions. The final `imsharpen` step adds brushstroke clarity characteristic of oil paintings. Notice how the skin looks very smooth and painted, but the eyes and lips are still clear.

7.4 Charcoal Sketch Filter

7.4 Charcoal Sketch Filter

PORTRAIT SAMPLE — CHARCOAL SKETCH EFFECT

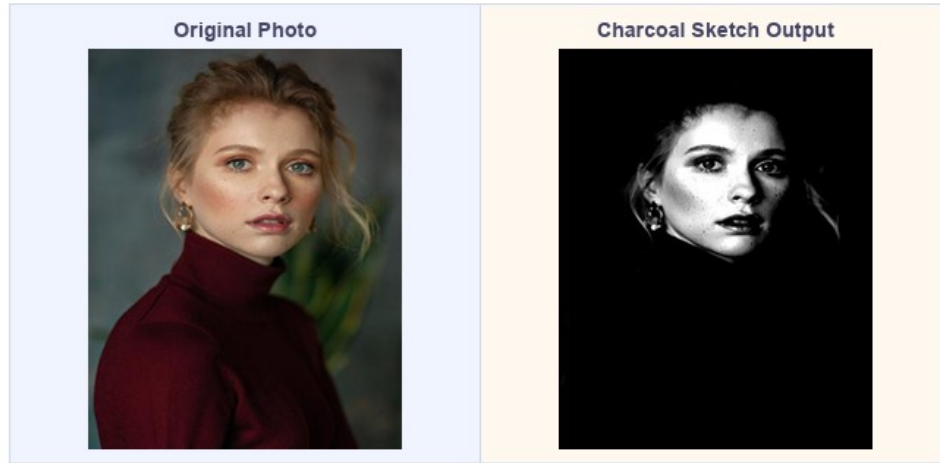


Figure 4: Testing the Charcoal Sketch Effect

The charcoal effect produces dramatic tonal contrast with deep shadows and highlighted areas. The texture map derived from `stdfilt` adds granularity that mimics the rough texture of charcoal on drawing paper, especially visible in hair and fabric regions. This filter gives a very moody and dark result.

7.5 Pop Art Filter

7.5 Pop Art Filter

PORTRAIT SAMPLE — POP ART EFFECT

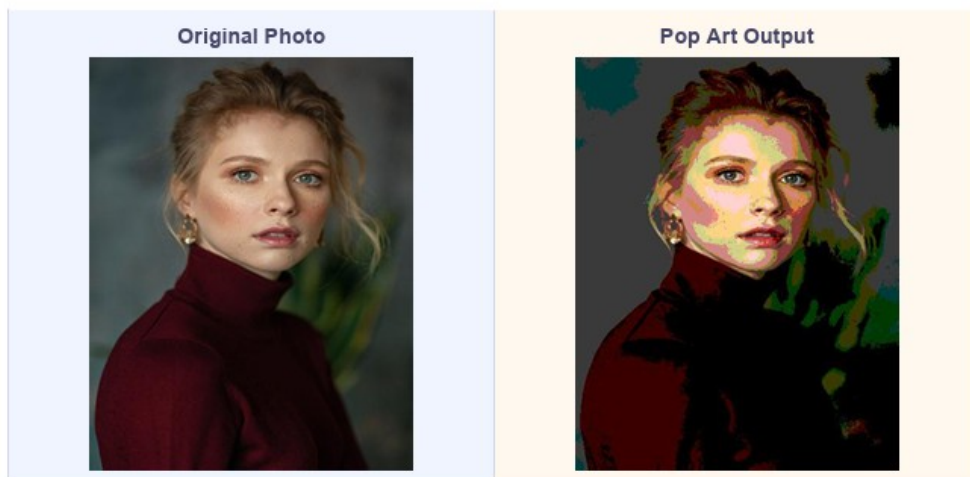


Figure 5: Testing the Pop Art Effect

By flattening the colors and increasing the contrast, we got a very bright, poster-like output with thick comic-style borders. The 5-level quantization creates the distinctive flat color zones of pop art while the black Canny outlines reinforce the comic-book aesthetic. The final contrast stretch ensures vivid, saturated colors that are visually striking and instantly recognizable as the pop art style.

7.6 Testing on Different Subjects

We wanted to ensure our app works on both male and female portraits with different lighting. The app works consistently well, handling different skin tones, hair types, and backgrounds without crashing or breaking the image.

7.6 Multi-Sample Comparison

DIFFERENT SUBJECTS — PENCIL SKETCH VS OIL PAINTING

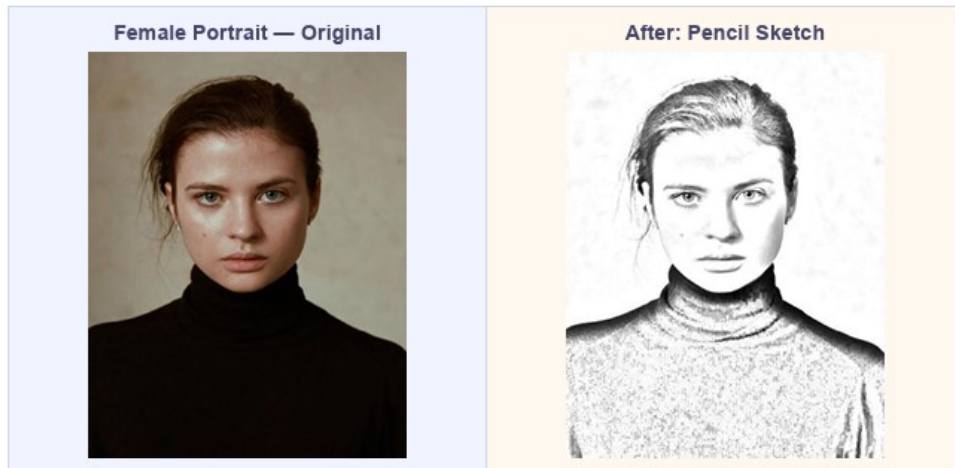


Figure 6: Female Subject — Pencil Sketch

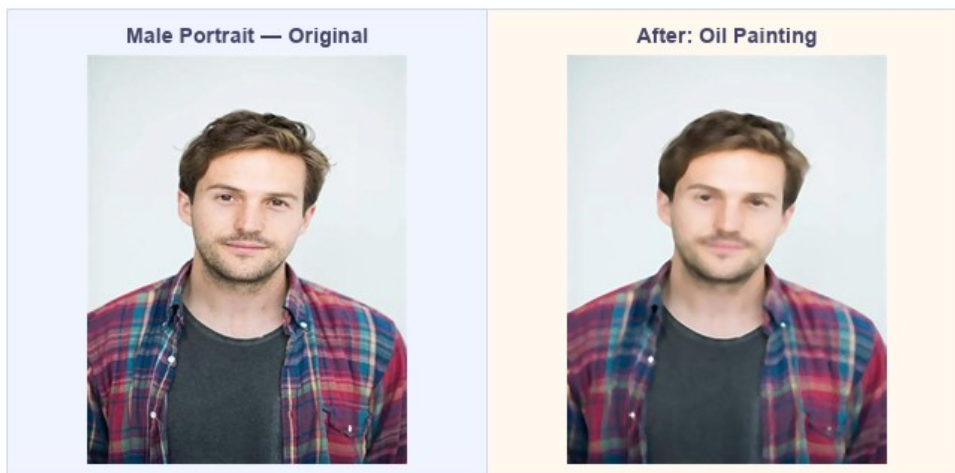


Figure 7: Male Subject — Oil Painting

The filters perform consistently across different subjects, lighting conditions, and facial structures. The pencil sketch preserves fine texture details while oil painting creates smooth, painterly regions in both female and male portrait subjects.

8 Code Structure & Good Practices

The entire application is implemented as a single MATLAB classdef file: `DIP.mlapp`. We tried to keep our MATLAB code very clean and organized so it is easy to read.

Section	Description
<code>properties</code> (<code>Access=public</code>)	Declares all 6 UI component handles: <code>UIFigure</code> , <code>DropDown</code> , <code>DropDownLabel</code> , <code>UploadImageButton</code> , <code>UIAxes</code> , <code>UIAxes2</code>
<code>DropDownValueChanged()</code>	Core processing callback: reads stored image → runs selected filter algorithm → displays result in <code>UIAxes2</code>
<code>UploadImageButtonPushed()</code>	File picker → <code>imread</code> → stores in <code>UserData</code> → displays in <code>UIAxes</code>
<code>createComponents()</code>	Initializes all UI elements with positions, colors, event bindings, and dropdown items
<code>DIP()</code> constructor	Entry point: calls <code>createComponents</code> → <code>registerApp</code> → returns app handle
<code>delete()</code>	Cleanup method: deletes <code>UIFigure</code> on app shutdown

Key Design Decisions:

- **Smart Storage:** Instead of creating new variables over and over, we stored the uploaded image in `UIFigure.UserData`. This avoided state management complexity and made our code much faster.
- **Double Precision:** Before doing any math (like dividing or multiplying pixels), we used `im2double`. All filter algorithms work on `[0,1]` double-precision arrays for mathematical stability. This prevented our image colors from breaking.
- **Switch-Case Routing:** Filter selection uses a clean switch-case structure, making it easy to add new artistic styles in future without changing the overall control flow.
- **Error Prevention (Guard Clause):** `DropDownValueChanged` returns immediately if `UserData` is empty. If a user tries to click a filter before uploading a photo, the app simply ignores the click instead of crashing and showing red errors.

9 Conclusion

Artistic Image Studio successfully demonstrates the practical application of Digital Image Processing concepts in a real-world interactive application. By combining classical DIP techniques — including Gaussian filtering, Canny edge detection, morphological operations, bilateral filtering, and intensity transformations — the application transforms ordinary photographs into visually compelling artistic renditions.

The project achieves all its stated objectives: five distinct artistic filters are implemented and functional, the MATLAB App Designer GUI provides an intuitive dual-panel interface for real-time comparison, and the underlying algorithms correctly apply DIP principles studied in the course. By carefully tweaking the numbers for blurring, contrast, and edge detection, we managed to make the outputs look very realistic. This project helped us see how the formulas we study in lectures are

actually used to build real-world software like Photoshop or Instagram filters.

The before-and-after results demonstrate that the filters produce consistently high-quality outputs across different input images, including varied lighting conditions, skin tones, and subject types. Each filter has been fine-tuned with appropriate parameters to maximize artistic resemblance to its namesake medium.

9.1 What We Can Add in the Future

If we had more time, we would love to add a few more things to this app:

- A "Save Image" button so users can download their new art to disk.
- Sliders so the user can manually control how dark or blurry they want the filter to be (e.g., sketch detail level, oil paint smoothness).
- Additional artistic styles such as Watercolor, Mosaic, HDR Tonemapping, or Vintage Film effects.
- A feature to apply filters to an entire folder of images at once (batch processing).
- A histogram display panel to visualize intensity distribution changes before and after processing.
- Undo/redo functionality for iterative artistic exploration.

References

- [1] Gonzalez, R. C., & Woods, R. E. *Digital Image Processing*, 4th Edition, Pearson (2018).
- [2] MATLAB Image Processing Toolbox Documentation, MathWorks (R2021a).
- [3] MATLAB App Designer Documentation, MathWorks: Getting Started with App Designer.
- [4] Canny, J. A Computational Approach to Edge Detection. *IEEE TPAMI*, 8(6), 679–698 (1986).
- [5] Tomasi, C. & Manduchi, R. Bilateral Filtering for Gray and Color Images. *ICCV* (1998).